Effect Boundary Lab: Retention-Aware Retry Safety Across a Local-Remote Commit Boundary

Vinay Muppalla | Independent technical report v1.0 | 23 October 2026

Status: public technical report; not peer reviewed. Source, data and reproduction instructions accompany the report.

Abstract

A durable local receipt does not make a remote effect atomic with local state. Stable destination keys address retries while their records remain available, but expiry changes the recovery contract. We present a small executable benchmark that contrasts naive replay, local receipts, destination keys and a conservative guarded retry policy. In 1,152 factorial trials with 1,008 real worker exits, the destination-key policy produced 36 excess effects in the expiry scenario. Guarded retry produced no excess effects or false confirmations in its 288 trials, but quarantined 72 operations, including 36 with no remote effect. A paired witness generates equal durable client state with different remote outcomes. The contribution is a reproducible empirical artifact and explicit accounting of safety versus completion, not a new idempotency primitive or a distributed exactly-once guarantee.

1. Motivation and research questions

Consider a worker that calls a second service, receives success and dies before recording the result. Its local database cannot reveal whether the external effect occurred. A local receipt helps only after it commits. A stable key at the destination helps only under that destination's retention and parameter-binding contract. These dependencies matter for software jobs, data connectors and state-changing agent tools.

We ask which crash windows defeat local receipts (RQ1), how destination receipt expiry changes replay safety (RQ2), what completion cost accompanies conservative quarantine (RQ3), and whether differing remote outcomes can leave equal durable client evidence (RQ4). The study reports these questions without treating a refusal to act as successful completion.

2. Prior work and contribution

Caller-supplied request identity and safe retries are established in AWS guidance [1]. Stripe documents request-parameter matching and API v1 key pruning after at least 24 hours [2]. RIFL coordinates durable RPC completion results at a much broader systems scale [3]. The idempotent consumer pattern couples processed-message records to a local business transaction [4]. Transactional outbox guidance addresses local event publication but still requires duplicate handling [5]. Temporal documents repeated activity execution and destination-enforced idempotency [6]. Existing public middleware also uses quarantine for uncertain effects [7]. A recent preprint describes related quarantine and crash experiments in distributed agent budget conservation [8]; only its abstract-level scope was reviewed here.

Our contribution is the integrated, inspectable fault matrix, semantic evidence verifier and executable equal-state/different-outcome witness. Neither quarantine nor retention-aware caution is claimed as new. This targeted review does not establish first-in-world novelty. No comparison with the performance or implementation quality of those systems is made.

3. Experimental system

A controller runs a threaded HTTP service on loopback. The destination has its own SQLite WAL database; each worker uses another local database. Every worker invocation is a new Python subprocess. An injected checkpoint calls os._exit(73). The harness rejects every unexpected return code. SQLite uses synchronous=FULL; integrity is checked after each trial. This tests worker-process failure rather than power loss.

The destination atomically appends an effect and optionally records a stable key, payload hash, result identity and creation time. It replays matching requests, rejects changed payloads with HTTP 409, and removes receipts at virtual age greater than or equal to ten units. Its effect table remains available to the audit controller, never to the recovering worker. The virtual clock and TTL are experimental controls, not a model of a vendor's exact cleanup timing.

Each trial contains one original logical operation, an initial invocation and a recovery invocation. Three synthetic payload labels denote payment, stock and notification examples. They all use the same append-effect implementation; there are no real charges, inventory updates or messages. Twelve fixed seeds vary the integer payload and are paired across conditions. The resulting design is 4 policies x 8 scenarios x 3 labels x 12 seeds = 1,152 trials.

4. Policies and failure schedules

Naive replay always dispatches without a destination key. Local receipt suppresses dispatch after a local result exists but has no destination key or local payload check. Destination key uses a stable identity with destination payload binding; it does not impose a client retention horizon. Guarded retry combines destination keys with local payload binding, cached results and an explicit stop when an unresolved intent reaches the known retention horizon.

The no-crash control still invokes the operation twice to test duplicate delivery. Other schedules exit before sending HTTP, after receiving external success, before local receipt commit, or after local receipt commit. Two delayed schedules retry at time twenty: one after an external effect, the other after intent persistence but before any effect. A changed-retry schedule increases the value under an already-used logical identity. It is intentionally invalid reuse rather than a legitimate second operation. Ordinary retries occur at time one.

The guard may replay a completed local receipt beyond the horizon because the effect identity is already known. It quarantines only unresolved expired intents. It cannot distinguish a never-sent request from an applied-but-unrecorded request using local intent state alone.

5. Measurement and verification

Excess effects equal max(number of effects minus one, zero) per original logical intent. Missing means no effect exists after recovery. Wrong payload counts effects that differ from the original intent. False confirmation means the worker claims a result identity whose stored payload does not match the retried request. Wrong payload and excess effects overlap; totals must not be added as disjoint incidents. Quarantine and conflict are explicit outcomes, not successful completion.

Raw evidence includes inputs, retry inputs, actual effect rows and hashes, worker outcome, request count, return-code checks and integrity results. A separate verifier, without importing the lab's metric implementation, recomputes effect-derived measurements, checks the full factorial grid and compares all 32 aggregate groups. Source SHA-256 binds the evidence to lab.py. Tests also alter a recorded metric and require verification to fail.

Twelve seeds are input variations, not independent samples of real failure frequency. The schedules are deterministic. We therefore report exact counts and do not attach inferential p-values or

production-reliability confidence bounds. Diagnostic wall time includes subprocess startup and is not a performance benchmark.

6. Results

Policy	Trials	Excess effects	Missing	False confirmations	Quarantined
Naive replay	288	216	0	0	0
Local receipt	288	108	0	36	0
Destination key	288	36	0	0	0
Guarded retry	288	0	36	0	72

The factorial run injected 1,008 worker exits; the no-crash cells account for the remaining initial invocations. All databases passed integrity checks. Each policy/scenario cell contains 36 trials.

For RQ1, the local-receipt baseline repeated effects after remote success and before durable local confirmation. It also falsely confirmed 36 changed-payload retries by returning an old result without binding the local record to the current request.

For RQ2, destination keys avoided in-window repeated effects in the evaluated schedules. After the destination removed its receipt, all 36 expired-after-effect trials executed a second effect. These results demonstrate the fixture's contract rather than a deficiency in a particular commercial API.

For RQ3, the guard prevented excess effects and false confirmations in its tested cells. Its 72 quarantines consist of 36 expired-after-effect cases with one existing effect, and 36 expiry-before-send cases with no effect. Consequently it prevented unsafe re-execution at the cost of leaving 36 requested effects unperformed. No policy dominates every safety and completion measure.

For RQ4, the witness generator terminates guarded workers before dispatch and after external success. Both persistent client rows contain the same payload hash, start time zero and absent effect identity. Their remote effect counts differ: zero and one. At time twenty both recoveries quarantine. This is a concrete ambiguity counterexample, not a new impossibility theorem.

7. Interpretation and application

A useful review question is not simply whether a queue or client is idempotent, but where the result and duplicate record are committed, how identity binds to payload, and how long that evidence remains trustworthy. A conservative stop makes uncertainty visible; it does not resolve uncertainty. Operational reconciliation needs authoritative destination evidence, a stable business identity, authentication, a recovery owner and a documented decision on whether to retry, accept or compensate.

The artifact is useful for empirical software testing and for discussing reliable data and agent infrastructure. The experiment does not run a language model or evaluate reasoning quality. Its lessons concern state-changing tools independent of the caller. Generalisation to a concrete data pipeline, healthcare integration or agent framework requires additional implementation and measurement.

8. Limitations and validity threats

The provider is a thread in the surviving controller, not an independently failing remote host. Actual response loss, server restart, network partition, power loss, disk corruption and clock skew are absent. Virtual time is supplied by the client fixture; a production horizon cannot trust arbitrary request timestamps. One logical operation and sequential clients simplify the concurrency problem. The separate 16-request concurrency test checks destination key arbitration only. Guarded clients have no lease, fencing or multi-worker coordination.

All three labelled workloads share one storage operation and must not be treated as independent domain validations. Fixed execution order, one local environment and synthetic payloads constrain external validity. The benchmark uses selected fault schedules rather than exhaustive state-space exploration. The verifier checks internal consistency; an independent external replication is still needed. Retention-aware rejection is an existing technique, and the targeted related-work search is not an exhaustive novelty review.

9. Reproduction and release status

With Python 3.10 or later, run lab.py run --seeds 12 --output evidence/results.json, then verify.py evidence/results.json, witness.py and python -m unittest -v test_lab from the artifact directory. No third-party Python package, account or external API is required. The repository includes source, raw JSON/CSV, protocol, tests, source hashes and a browser evidence explorer. The parent project's queue tests are run separately.

This v1.0 is released as an open technical report. It is not an accepted paper, a production-ready payment service, or proof of an unrestricted distributed guarantee. Subsequent research should evaluate authenticated reconciliation, realistic time and provider failures, multiple workers, and externally replicated workloads before making broader claims.

References

[1] M. Featonby. Making retries safe with idempotent APIs. AWS Builders' Library. https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

[2] Stripe. Idempotent requests. API reference. https://docs.stripe.com/api/idempotent_requests

[3] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita and J. Ousterhout. Implementing Linearizability at Large Scale and Low Latency. SOSP, 2015. DOI: 10.1145/2815400.2815416.

[4] C. Richardson. Pattern: Idempotent Consumer. https://microservices.io/patterns/communication-style/idempotent-consumer.html

[5] AWS. Transactional outbox pattern. Prescriptive Guidance. https://docs.aws.amazon.com/en_en/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[6] Temporal. Activity Definition. https://github.com/temporalio/documentation/blob/main/docs/encyclopedia/activities/activity-definition.mdx

[7] swarmproof. exactly-once. Public software repository. https://github.com/swarmproof/exactly-once

[8] G. Zhu and C. Wang. Fault-Tolerant Budget Conservation in Distributed Multi-Agent Delegation. arXiv:2610.00349, 2026. Preprint; abstract-level scope reviewed. https://arxiv.org/abs/2610.00349

Web references accessed 23 October 2026. Code and report: https://github.com/vinaychowdary9946-blip/resilient-job-queue/tree/main/research/effect-boundary-lab